Del simulador al mapa

SLAM en ROS 2 con TurtleBot3 y Gazebo

Jhon Hesneydert Londoño · Juan David Villota

Robótica — Capítulo 6 · Parte 8 · UNAL Manizales

github.com/juan-villota-b

Las preguntas de esta simulación

`simple_controller` de la guía: publica 0.2 m/s y 0.1 rad/s a 10 Hz. No se suscribe a nada.

¿Y si choca?
Sigue mandando la misma velocidad. El plugin retiene el último comando: no hay *timeout*.

¿Y si se vuelca?
Las ruedas giran en el aire, la odometría reporta avance y RViz sigue "mapeando" un robot caído.

¿Cubre el mapa?
El círculo de 2 m se queda en una franja: la arena mide 5.6 m y es hexagonal.

Medido antes de tocar nada
pitch oscilando 75° → 89° (rango 14.75°) · giroscopio en 0 con comando de 0.5 rad/s · altura del modelo 0.038 m

90°
robot volcado

Con el LiDAR mirando al piso, cada barrido era un corte distinto del mundo: el mapa no podía salir bien.

El LiDAR mide un plano

El LDS-01 lanza el láser en **un plano horizontal** a 17 cm del piso y devuelve 360 distancias. No hay altura ni volumen: es **una rebanada** del mundo.

Si el robot se inclina un ángulo ϕ, el haz que va a distancia d se desvía:

$$d \tan \phi$$

Y el piso se vuelve un obstáculo fantasma a:

$$d = \frac{h}{\tan \phi} \approx 1.4 \text{ m} \quad (\phi = 7^\circ)$$

Distancia	Desvío a 7°
1 m	12 cm
2 m	25 cm
3.5 m	**43 cm**

El *scan matching* solo puede trasladar y girar: **no tiene variable para la inclinación**. Con cada barrido distinto elige la transformación “menos mala” y el mapa se ensucia.

Modificación 1 · La física del robot

El modelo oficial de Gazebo deja el caster **1 mm en el aire**: el robot se sostiene en dos ruedas y se mece.

Archivos nuevos
`models/turtlebot3_burger/model.sdf`
`launch/lab_world.launch.py`

Parámetro	Antes	Ahora
caster `z`	−0.004	**−0.006**
ruedas `mu`	100 000	**1.0**
`min_depth`	0.001	**0.0**

14.75°
balanceo antes

0.00°
después

Con el caster apoyado y fricción real, el plano del LiDAR queda horizontal y **cada barrido vuelve a ser válido**.

Del lazo abierto al seguidor de pared

El controlador de la guía no controla: **genera comandos**. Para mapear hacía falta algo que mirara el entorno.

La regla es mantener la pared de la derecha a 0.9 m:

$$\omega = -k\,(d - 0.9)$$

con ω saturado a ± 0.35 rad/s.

- Si el frente ve menos de 0.45 m: gira en el lugar.
- Si ve menos de 1.0 m: baja a 0.06 m/s.

```
right = self.right_range
error = right - self.wall_distance
omega = max(-0.35, min(0.35, -0.9 * error))

speed = 0.14 if self.front_range > 1.0 else 0.06
self.publish(speed, omega)
```

Mide con `/odom`, no con el reloj, y se adapta a la forma real de la arena en vez de seguir coordenadas fijas.

Modificación 3 · Cartographer no está: SLAM Toolbox

El problema de empaquetado
Para *lyrical* solo existe `cartographer-ros-msgs`. No hay `cartographer_ros`: los pasos 8.5, 8.13 y 8.21 no se pueden ejecutar.

La alternativa
`ros-lyrical-slam-toolbox` 2.10.0: grafo de poses + matcher Karto + optimizador Ceres. Modo *online* y *lifelong*.

La interfaz observable es la misma: `/map`, TF `map→odom` y el mismo `map_saver_cli`.

	Cartographer	SLAM Toolbox
Origen	Google	comunidad ROS 2
Estructura	submapas + grafo	poses + barridos
Matcher	correlación *branch-and-bound*	Karto
Config	`.lua`	`.yaml`
Extra	`/submap_list`	grafo en MarkerArray

Qué se conservó: `cartographer.lua` y `cartographer.launch.py` quedan en el paquete para reproducir la guía literal en Jazzy. El `.yaml` replica sus números: 0.05 m/celda, rango 0.12–3.5 m, frames `map` / `odom` / `base_footprint`.

Además: RViz sin el panel `Displays` y Gazebo sin `EntityTree`, para que la grabación muestre solo el mapa y el mundo.

Las 5 terminales y el paquete

Qué corre en cada una

T1 · Gazebo — `ros2 launch localization_lab lab_world.launch.py`
Simula el mundo y el burger; publica `/scan`, `/odom` y `/clock`.

T2 · SLAM — `ros2 launch localization_lab slam_toolbox.launch.py`
Alinea barridos y publica `/map` + TF `map→odom`.

T3 · Control — `ros2 run localization_lab patrol_controller`
Lee `/scan` y `/odom`; decide la velocidad y la publica en `/cmd_vel`.

T4 · RViz2 — `rviz2 -d .../rviz/slam.rviz`
Dibuja el mapa, el barrido y los frames; no calcula nada.

T5 · Mapa — `map_saver_cli -f mapa`
Congela `/map` en `.pgm` + `.yaml`.

Cómo está organizado el paquete

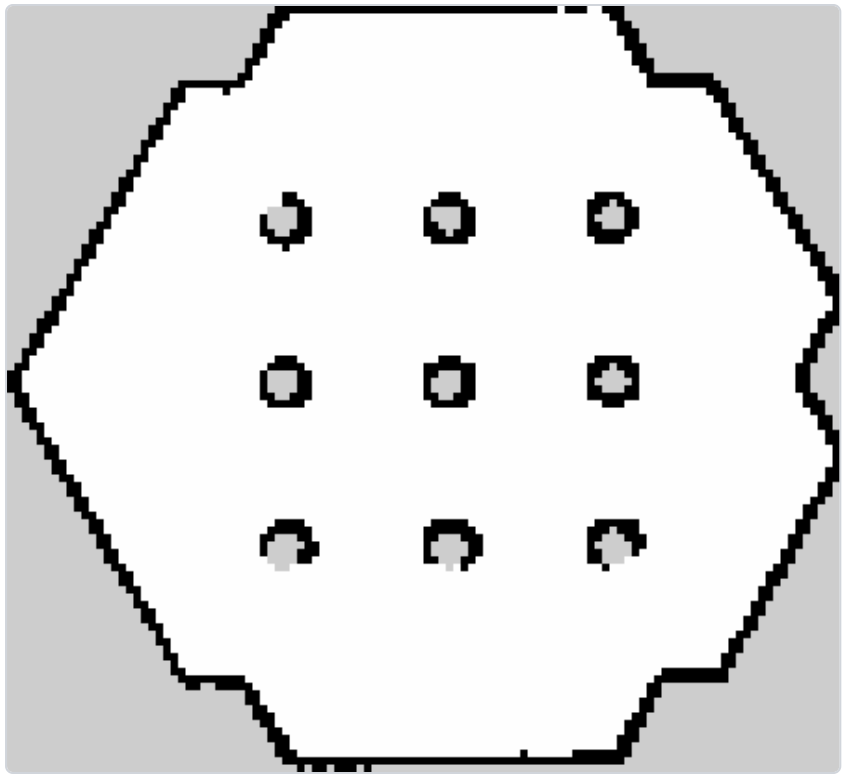
```
localization_lab/
├── package.xml          dependencias
├── setup.py             entry points
├── localization_lab/
│   ├── simple_controller.py    la guía
│   └── patrol_controller.py    seguidor
├── launch/   lab_world · slam_toolbox
├── config/   cartographer.lua · slam_toolbox.yaml
├── models/   turtlebot3_burger
└── rviz/     slam.rviz
```

Cada nodo declara sus parámetros (`linear_speed`, `wall_distance`, `duration`), así el comportamiento se cambia sin editar código.

Un solo `colcon build --symlink-install` compila el paquete y deja los dos ejecutables.

Resultado · el mapa

Salida de `map_saver_cli` + conversión a PNG



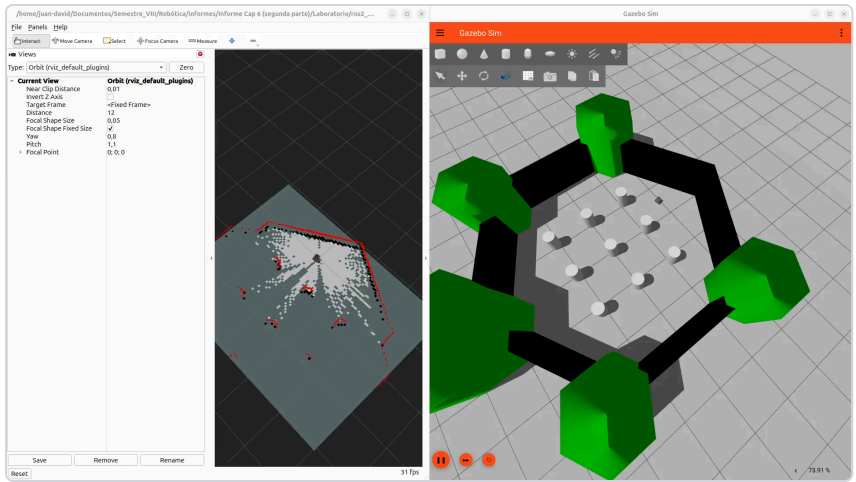
112 × 103
celdas

5.6 × 5.15 m
a 0.05 m/celda

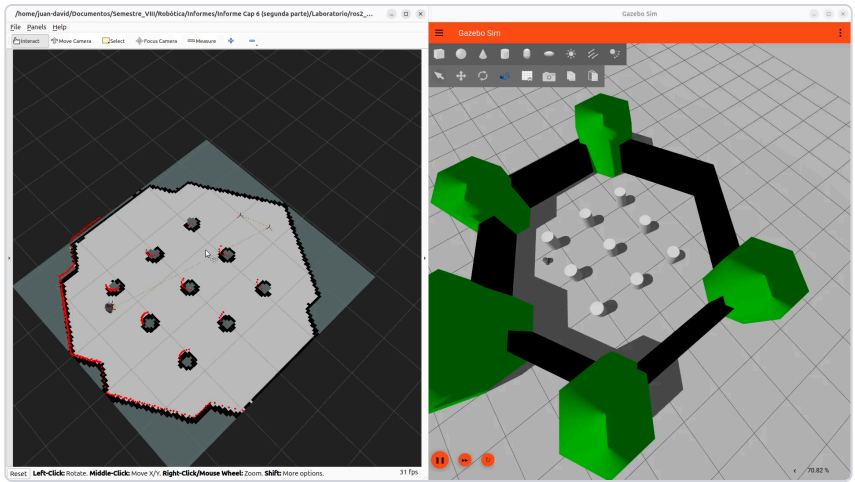
- Hexágono cerrado, sin paredes dobles.
- Los **9 cilindros** del centro en su grilla de 3 × 3.
- Recorrido por el anillo libre, sin tocar obstáculos.

El mapa es el insumo de AMCL y Nav2: cierra el ciclo del capítulo.

Arranque y final de la corrida



Primeros barridos: solo el sector que el LiDAR alcanza a ver.



Al detenerse: la arena completa, con paredes y cilindros.

> Lo que cambia entre las dos no es el algoritmo: es la **cobertura**. El primer barrido ya es válido porque el plano del LiDAR está horizontal.

Respuestas · problemas y soluciones

Problema	Causa	Solución
`cartographer_ros` no instala	no está publicado para *lyrical*	SLAM Toolbox, mismo `/map` y TF `map→odom`
El robot se mecía y se volcaba	caster 1 mm en el aire + fricción infinita	modelo propio: caster −0.006 y `mu` 1.0
El mapa salía borroso	plano del LiDAR inclinado	consecuencia del anterior
El cuadrado no cubría la arena	arena hexagonal de 5.6 m	seguidor de pared con `/scan`
El robot no se movía	simulación en pausa: `/clock` no avanzaba	reanudar con `/world/default/control`
Se cerraba todo al cerrar Gazebo	`on_exit_shutdown` en el cliente	no cerrar la GUI durante la demo
`ros2 run` no encontraba el nodo	`console_scripts` vacío en la guía	entry point en `setup.py`

Todas las diferencias están documentadas en el README del laboratorio, con el número de anomalía.

Demostración en video

Recorrido completo del seguidor de pared, con RViz y Gazebo en paralelo.

▶️ Ver grabación

youtu.be/JArwIEL_Rsk

240 s de recorrido · 4:27 de video · mapa guardado en `.pgm` + `.yaml`

Conclusiones

01

No se depura el algoritmo: se depura la **suposición física**. Un LiDAR 2D exige un plano horizontal.

02

El control de la guía es de **lazo abierto**. Cambiarlo por uno que mire el sensor permitió recorrer toda la arena.

03

El mapa es una **estimación**: se corrige, se ensucia y se reacomoda. Guardarlo bien es lo que lo vuelve útil.

github.com/juan-villota-b/Robotica-unal-informes